

# Brainbase Model Runtime — Final Coding Agent Instruction Document

## 0. Mission

Build a backend-only MVP for the **Brainbase Model Runtime**.

This product exposes Brainbase-owned model names to customers:

```
brainbase-chat  
brainbase-code  
brainbase-agent  
brainbase-fast  
brainbase-premium
```

Customers should call it like a normal OpenAI-compatible API.

Internally, the system should route each request through a stable runtime:

```
OpenAI-compatible API  
  → RouterPolicy  
  → RoutePlan  
  → RoutePlan Validator  
  → Model Resolver  
  → Portkey  
  → OpenRouter  
  → Response  
  → Supabase trace
```

The product is not one router implementation. It is a **runtime shell** that allows future routers, models, and trained artifacts to plug in.

The MVP is complete when:

```
New model:  
  add model string/config → test → calibrate → enable  
  
New router:  
  implement RouterPolicy.plan() → benchmark cheaply → shadow → promote
```

```
Customer:
  calls model="brainbase-chat"
```

No frontend is required.

---

## 1. Non-Negotiable Rules

Do not build a frontend.

Do not expose the word "router" in public customer-facing model names.

Do not hardcode one provider or one model throughout the codebase.

Do not build the product around HyDRA, GraphRouter, Avengers-Pro, RouteLLM, or any single algorithm.

Do not add unnecessary infra:

```
No LangFuse
No Sentry
No Redis
No Kafka
No Stripe
No Clerk
No direct OpenAI/Anthropic/Gemini keys for MVP
No S3 unless Supabase Storage becomes insufficient
```

Do not run expensive benchmarks during development.

Do not run full benchmarks automatically.

Do not run real provider calls inside normal test suites.

All normal tests must work without paid external model calls.

---

## 2. Required MVP Stack

Use:

```
Backend:
  Python
```

FastAPI  
Pydantic  
Uvicorn

Execution:

Portkey

Model access:

OpenRouter through Portkey

Database:

Supabase Postgres

Artifact/report storage:

Supabase Storage

Deployment:

Render

Version control:

GitHub

Public API style:

OpenAI-compatible chat completions

---

## 3. Required Environment Variables

### Required for core MVP

```
PORTKEY_API_KEY=  
OPENROUTER_API_KEY=  
  
SUPABASE_URL=  
SUPABASE_SECRET_KEY=  
SUPABASE_PUBLISHABLE_KEY=
```

Legacy Supabase equivalents are acceptable:

```
SUPABASE_URL=  
SUPABASE_SERVICE_ROLE_KEY=  
SUPABASE_ANON_KEY=
```

## Required only if deploying programmatically

```
RENDER_API_KEY=
```

## Required only if coding agent needs repo access

```
GITHUB_TOKEN=
```

## Local mock development mode

```
EXECUTOR_MODE=mock  
REGISTRY_MODE=yaml  
STORAGE_MODE=memory
```

## Supabase integration mode

```
EXECUTOR_MODE=mock  
REGISTRY_MODE=supabase  
STORAGE_MODE=supabase
```

## Real provider smoke mode

```
EXECUTOR_MODE=portkey  
REGISTRY_MODE=supabase  
STORAGE_MODE=supabase  
RUN_REAL_PROVIDER_TESTS=true
```

---

## 4. Required Repo Layout

Create this structure:

```
brainbase-model-runtime/  
  README.md  
  Makefile  
  pyproject.toml  
  Dockerfile
```

```
render.yaml
.env.example

app/
  main.py
  config.py
  dependencies.py

api/
  openai_compatible.py
  models.py
  health.py
  internal.py

protocol/
  router_request.py
  model_profile.py
  routing_context.py
  routing_budget.py
  route_plan.py
  route_step.py
  feedback_event.py
  schemas.py

runtime/
  kernel.py
  request_normalizer.py
  public_model_resolver.py
  policy_loader.py
  model_candidate_resolver.py
  route_plan_validator.py
  fallback_manager.py
  response_normalizer.py

registries/
  model_registry.py
  policy_registry.py
  tenant_registry.py
  model_registry.yaml
  policy_registry.yaml
  tenant_config.yaml

policies/
  base.py
  always_strongest/
    policy.py
    tests.py
  always_cheapest/
```

```
    policy.py
    tests.py
always_fastest/
    policy.py
    tests.py
manual_rules/
    policy.py
    tests.py
random_policy/
    policy.py
    tests.py

executor/
    mock_executor.py
    portkey_executor.py
    openrouter_model_resolver.py
    execution_result.py
    errors.py

storage/
    supabase_client.py
    trace_repository.py
    registry_repository.py
    benchmark_repository.py
    artifact_repository.py

database/
    supabase_schema.sql

evals/
    benchmark_runner.py
    replay_runner.py
    policy_comparator.py
    scoring.py
    report_generator.py

sample_datasets/
    basic_prompts.jsonl
    code_prompts.jsonl
    reasoning_prompts.jsonl
    agent_tool_prompts.jsonl
    support_prompts.jsonl

scripts/
    seed_registry.py
    benchmark_policy.py
    test_model_string.py
    smoke_mock.py
```

```
smoke_real.py

docs/
  customer-usage.md
  internal-router-policy.md
  add-new-model.md
  add-new-policy.md
  testing.md
  benchmarking.md
  deployment.md

tests/
  protocol/
  api/
  runtime/
  registries/
  executor/
  storage/
  policies/
  evals/
```

---

## 5. Core Protocol Contract

Every router algorithm must implement one interface:

```
class RouterPolicy:
    name: str
    version: str
    supported_modes: list[str]

    def plan(
        self,
        request: RouterRequest,
        candidates: list[ModelProfile],
        context: RoutingContext,
        budget: RoutingBudget
    ) -> RoutePlan:
        ...
```

Routers must only return a `RoutePlan`.

Routers must not call providers directly.

Routers must not know Portkey headers, OpenRouter API syntax, Supabase, or HTTP details.

---

## 6. Core Protocol Objects

Implement these with Pydantic.

### RouterRequest

```
class RouterRequest(BaseModel):
    request_id: str
    public_model: str
    messages: list[dict]
    tools: list[dict] | None = None
    response_format: dict | None = None
    modality: str = "text"
    tenant_id: str | None = None
    workflow_id: str | None = None
    step_id: str | None = None
    metadata: dict = {}
```

### ModelProfile

```
class ModelProfile(BaseModel):
    id: str
    executor: str
    executor_model: str
    provider: str
    status: str
    supports: dict
    limits: dict
    cost: dict
    capabilities: dict
    metadata: dict = {}
```

### RoutingContext

```
class RoutingContext(BaseModel):
    tenant_config: dict
    public_model_config: dict
    workflow_context: dict | None = None
```



```
historical_performance: dict | None = None
policy_config: dict = {}
```

## RoutingBudget

```
class RoutingBudget(BaseModel):
    max_cost_usd: float | None = None
    max_latency_ms: int | None = None
    max_cascade_depth: int = 1
    allow_multi_call: bool = False
    allow_expensive_models: bool = False
    mode: str = "balanced"
```

## RoutePlan

```
class RoutePlan(BaseModel):
    mode: str
    selected_model: str | None = None
    steps: list[RouteStep] = []
    fallback_models: list[str] = []
    confidence: float = 1.0
    policy_name: str
    policy_version: str
    metadata: dict = {}
```

Supported MVP modes:

```
single
cascade
agent_step
```

Future modes can exist in schema but do not need full execution yet:

```
multi_call
budgeted_single
context_routing
```

## 7. Public API Requirements

Expose:

```
GET /health
GET /v1/models
POST /v1/chat/completions
```

### /v1/models

Must return public Brainbase model names:

```
brainbase-chat
brainbase-code
brainbase-agent
brainbase-fast
brainbase-premium
```

### /v1/chat/completions

Must accept an OpenAI-style payload:

```
{
  "model": "brainbase-chat",
  "messages": [
    {
      "role": "user",
      "content": "Say hello in one sentence."
    }
  ]
}
```

Must return an OpenAI-compatible response:

```
{
  "id": "chatcmpl_...",
  "object": "chat.completion",
  "created": 1234567890,
  "model": "brainbase-chat",
  "choices": [
    {
```

```
    "index": 0,
    "message": {
      "role": "assistant",
      "content": "...",
    },
    "finish_reason": "stop"
  }
],
"usage": {
  "prompt_tokens": 0,
  "completion_tokens": 0,
  "total_tokens": 0
}
}
```

Important:

The public response model must remain brainbase-chat.  
Do not expose the internal selected provider model unless debug mode is explicitly enabled.

---

## 8. Runtime Flow

Implement this exact flow:

1. Receive OpenAI-compatible request.
2. Normalize request into RouterRequest.
3. Resolve public model alias.
4. Load RoutingContext.
5. Load RoutingBudget.
6. Load candidate ModelProfiles from model registry.
7. Load active RouterPolicy from policy registry.
8. Call policy.plan(...).
9. Validate RoutePlan.
10. Resolve selected model string into executor model.
11. Execute through mock executor or Portkey executor.
12. Normalize response into OpenAI-compatible response.
13. Store trace.
14. Return response.

## 9. Model String Format

Support model IDs like:

```
openrouter:anthropic/claude-sonnet
openrouter:openai/gpt-4o
openrouter:qwen/qwen-coder
openrouter:google/gemini-pro
anthropic:claude-opus-latest
openai:gpt-latest
selfhosted:brainbase-qwen-agent-v1
```

For MVP, real execution only needs OpenRouter through Portkey.

Example resolver behavior:

```
Input:
  openrouter:anthropic/claude-sonnet

Resolved executor model:
  @openrouter/anthropic/claude-sonnet
```

Routers output model IDs. The executor handles provider-specific formatting.

---

## 10. RoutePlan Validator

Before execution, validate every `RoutePlan`.

Checks:

```
selected_model exists for mode=single
selected_model exists in registry or is allowed dynamic passthrough
model is enabled
model belongs to allowed candidate pool
model supports requested tools/vision/json
model fits context limits
model fits budget
fallback models are valid
cascade depth is allowed
```

```
route mode is supported
policy is allowed for public model
```

If validation fails:

```
Do not execute the bad plan.
Use safe fallback policy or return controlled error.
Log validation failure.
```

---

## 11. Supabase Tables

Create:

```
public_model_aliases (
  id uuid primary key,
  public_model text,
  policy_name text,
  policy_version text,
  model_pool text,
  mode text,
  config jsonb,
  created_at timestamp
);

model_registry (
  id text primary key,
  executor text,
  executor_model text,
  provider text,
  status text,
  supports jsonb,
  limits jsonb,
  cost jsonb,
  capabilities jsonb,
  metadata jsonb,
  created_at timestamp,
  updated_at timestamp
);

policy_registry (
  id uuid primary key,
  name text,
  version text,
```

```

    type text,
    artifact_uri text,
    status text,
    supported_modes jsonb,
    config jsonb,
    created_at timestamp,
    updated_at timestamp
);

traces (
    request_id text primary key,
    public_model text,
    tenant_id text,
    policy_name text,
    policy_version text,
    route_plan jsonb,
    selected_model text,
    candidate_models jsonb,
    fallback_used boolean,
    input_tokens int,
    output_tokens int,
    cost_usd numeric,
    latency_ms int,
    status text,
    error text,
    created_at timestamp
);

benchmark_runs (
    id uuid primary key,
    name text,
    policy_name text,
    policy_version text,
    dataset_name text,
    mode text,
    metrics jsonb,
    report jsonb,
    created_at timestamp
);

router_artifacts (
    id uuid primary key,
    policy_name text,
    policy_version text,
    artifact_uri text,
    manifest jsonb,
    eval_report jsonb,
    status text,

```

```
    created_at timestamp
);
```

## 12. Required Test Modes

The product must support four test modes.

### 12.1 Mock mode

Purpose:

```
Fast local development.
No external APIs.
No paid model calls.
```

Env:

```
EXECUTOR_MODE=mock
REGISTRY_MODE=yaml
STORAGE_MODE=memory
```

This must test the complete runtime shell without providers.

### 12.2 Local YAML mode

Purpose:

```
Test local registry configs and routing behavior.
```

Env:

```
EXECUTOR_MODE=mock
REGISTRY_MODE=yaml
STORAGE_MODE=memory
```

### 12.3 Supabase integration mode

Purpose:

Test real database reads/writes without model execution.

Env:

```
EXECUTOR_MODE=mock  
REGISTRY_MODE=supabase  
STORAGE_MODE=supabase  
RUN_SUPABASE_TESTS=true
```

## 12.4 Real provider smoke mode

Purpose:

Run one or a few real Portkey/OpenRouter calls.  
Prove real execution works.  
Keep cost tiny.

Env:

```
EXECUTOR_MODE=portkey  
REGISTRY_MODE=supabase  
STORAGE_MODE=supabase  
RUN_REAL_PROVIDER_TESTS=true
```

This mode must never run automatically in normal tests.

---

## 13. Required Makefile Commands

Implement these:

```
make install
```

Install dependencies.

```
make test
```

Run all unit tests and non-paid integration tests.



```
make lint
```

Run linting.

```
make typecheck
```

Run type checks.

```
make smoke-mock
```

Start server in mock mode and hit `/v1/chat/completions`.

```
make smoke-real
```

Run one real Portkey/OpenRouter call. Must require `RUN_REAL_PROVIDER_TESTS=true`.

```
make benchmark-mock
```

Run benchmark pipeline with mock executor.

```
make benchmark-dry-run
```

Run benchmark routing decisions only, with no model execution.

```
make benchmark-canary-real
```

Run a tiny real benchmark with strict cost/call caps. Must require `RUN_REAL_PROVIDER_TESTS=true`.

```
make check
```

Run:

```
lint
typecheck
test
smoke-mock
```

```
benchmark-mock
benchmark-dry-run
```

Do not include `smoke-real` or `benchmark-canary-real` in `make check`.

---

## 14. Required Testing Per Module

### 14.1 Protocol tests

Must test:

```
valid RouterRequest
valid ModelProfile
valid RoutePlan
invalid RoutePlan
selected_model required for mode=single
cascade mode requires steps
policy_name required
policy_version required
```

Command:

```
make test tests/protocol
```

### 14.2 API tests

Must test:

```
GET /health
GET /v1/models
POST /v1/chat/completions
bad request handling
API delegates to runtime
OpenAI-compatible response shape
```

Use FastAPI TestClient.

Command:

```
make test tests/api
```

## 14.3 Registry tests

Must test:

```
YAML model registry loads
YAML policy registry loads
tenant/public model aliases load
candidate filtering by model_pool
disabled models excluded
unknown public model returns clear error
dynamic OpenRouter model string parsing
```

Command:

```
make test tests/registries
```

## 14.4 Executor tests

Must test:

```
mock executor returns valid ExecutionResult
OpenRouter model string resolver works
invalid model string errors
Portkey executor with mocked HTTP client
provider error handling
fallback execution skeleton
```

Normal executor tests must not call real providers.

Command:

```
make test tests/executor
```

Optional real provider test:

```
RUN_REAL_PROVIDER_TESTS=true make smoke-real
```

## 14.5 Storage tests

Must test:

```
Supabase client initializes from env
trace insert with mocked client
model registry read/write with mocked client
policy registry read/write with mocked client
benchmark insert with mocked client
schema file exists
```

Normal storage tests must not require real Supabase.

Optional Supabase test:

```
RUN_SUPABASE_TESTS=true make test tests/storage
```

## 14.6 Runtime tests

Must test:

```
request normalization
public model alias resolution
candidate model resolution
policy loading
valid RoutePlan passes validator
invalid selected_model fails
disabled model fails
unsupported tools/vision/json fails
end-to-end runtime with mock executor
trace object produced
fallback policy used when active policy throws
```

Command:

```
make test tests/runtime
make smoke-mock
```

## 14.7 Policy tests

Must test:

```
AlwaysStrongestPolicy
AlwaysCheapestPolicy
AlwaysFastestPolicy
ManualRulesPolicy
RandomPolicy
empty candidate list
disabled candidates
valid RoutePlan returned
policy_name/policy_version included
```

Command:

```
make test tests/policies
```

## 14.8 Benchmark tests

Must test:

```
dataset loading
mock benchmark execution
dry-run benchmark execution
policy comparison output
report JSON generation
benchmark result storage with mocked repository
shadow policy does not execute model
```

Command:

```
make test tests/evals
make benchmark-mock
make benchmark-dry-run
```

---

## 15. Benchmark Rules

Do not run full benchmarks during MVP development.

The benchmark system must be ready to run full benchmarks later, but development should only prove wiring and safety.

Implement four benchmark modes:

```
mock-benchmark  
dry-run-benchmark  
canary-real-benchmark  
full-benchmark
```

## 15.1 Mock benchmark

Runs benchmark pipeline with mock responses.

Cost:

\$0

Command:

```
python scripts/benchmark_policy.py  
--dataset sample_datasets/basic_prompts.jsonl  
--policies always-strongest,always-cheapest  
--executor mock
```

Must prove:

```
dataset loads  
policies run  
RoutePlans generate  
reports generate  
results store
```

## 15.2 Dry-run benchmark

Runs routing only. No model calls.

Cost:

\$0

Command:

```
python scripts/benchmark_policy.py
--dataset sample_datasets/realistic_prompts.jsonl
--policies always-strongest,always-cheapest
--dry-run
```

Must prove:

```
policies select models
RoutePlans validate
cost estimates work
reports generate
```

## 15.3 Canary real benchmark

Runs only one or two rows per dataset with real Portkey/OpenRouter calls.

Cost:

```
cents only
```

Command:

```
RUN_REAL_PROVIDER_TESTS=true python scripts/benchmark_policy.py
--datasets sample_datasets/code_prompts.jsonl,sample_datasets/
reasoning_prompts.jsonl,sample_datasets/agent_tool_prompts.jsonl
--policies always-strongest,always-cheapest
--limit-per-dataset 2
--max-real-calls 6
--max-cost-usd 0.50
--executor portkey
```

Required safety flags:

```
--limit-per-dataset
--max-real-calls
--max-cost-usd
--timeout-seconds
```

If estimated or actual cost exceeds the cap, stop immediately.

## 15.4 Full benchmark

Manual only.

Must require:

```
--full
--confirm-cost
```

Command:

```
python scripts/benchmark_policy.py
--datasets all
--policies always-strongest,always-cheapest,hydra,brainbase-trained
--executor portkey
--full
--confirm-cost
```

Without `--full` and `--confirm-cost`, the runner must refuse large jobs.

---

## 16. Required Sample Datasets

Create realistic but small JSONL datasets.

```
sample_datasets/
basic_prompts.jsonl
code_prompts.jsonl
reasoning_prompts.jsonl
agent_tool_prompts.jsonl
support_prompts.jsonl
```

Example row:

```
{
  "id": "code_001",
  "public_model": "brainbase-code",
  "messages": [
    {
      "role": "user",
      "content": "Debug this Python error: TypeError: 'NoneType' object is not
```



```
subscriptable."  
  }  
],  
"category": "coding",  
"difficulty": "easy",  
"expected_behavior": "Identify the None value source and suggest a fix."  
}
```

The canary benchmark should only execute one or two rows from each dataset.

---

## 17. Shadow Testing

Implement shadow testing skeleton.

Flow:

```
live policy executes  
shadow policy produces RoutePlan only  
shadow RoutePlan is validated  
shadow decision is logged  
shadow model is not executed by default
```

Test requirement:

```
A shadow policy must never call the executor unless explicitly enabled.
```

---

## 18. Parallel Development Plan

### Wave 0 — Foundation

One agent creates:

```
repo skeleton  
pyproject.toml  
Makefile  
.env.example  
Dockerfile  
render.yaml  
test setup
```

```
empty modules
README stub
```

Must pass:

```
make install
make test
make check
```

## Wave 1 — Protocol

One agent creates:

```
protocol/
policies/base.py
tests/protocol/
```

Must pass:

```
make test tests/protocol
```

No other agent should invent schemas.

## Wave 2 — Parallel Modules

After protocol lands, run these in parallel:

```
Agent B: API layer
Agent C: Supabase storage
Agent D: Registries
Agent E: Executor
Agent G: Baseline policies
```

Each agent must add module tests.

## Wave 3 — Runtime Integration

Runtime agent connects:

```
API request
registry
policy
validator
executor
storage
response
```

Must pass:

```
make test tests/runtime
make smoke-mock
```

## Wave 4 — Benchmark and Shadow

Benchmark agent builds:

```
benchmark runner
dry-run mode
mock mode
canary-real mode
full mode with confirmation guard
shadow testing skeleton
```

Must pass:

```
make benchmark-mock
make benchmark-dry-run
```

## Wave 5 — Deployment and Docs

Deployment/docs agent finishes:

```
Dockerfile
render.yaml
README
docs/testing.md
docs/benchmarking.md
docs/deployment.md
```

```
docs/add-new-model.md
docs/add-new-policy.md
```

Must pass:

```
make check
```

## 19. Step-by-Step Agent Prompts

### Foundation Agent Prompt

Create the repository skeleton for the Brainbase Model Runtime.

Add:

- pyproject.toml
- Makefile
- .env.example
- Dockerfile
- render.yaml
- README.md
- base folder structure
- pytest setup
- placeholder tests

Implement required Makefile commands:

```
make install
make test
make lint
make typecheck
make smoke-mock
make smoke-real
make benchmark-mock
make benchmark-dry-run
make benchmark-canary-real
make check
```

Do not implement product logic yet.

Acceptance:

```
make install passes
```

```
make test passes
make check passes
```

## Protocol Agent Prompt

Build the protocol package and RouterPolicy base interface.

Implement:

RouterRequest

ModelProfile

RoutingContext

RoutingBudget

RouteStep

RoutePlan

FeedbackEvent

RouterPolicy base class

Add validation tests.

No external APIs.

Acceptance:

make test tests/protocol passes

## API Agent Prompt

Build the OpenAI-compatible API.

Expose:

GET /health

GET /v1/models

POST /v1/chat/completions

The API must delegate to RuntimeKernel.

No routing logic in API.

No provider logic in API.

Add FastAPI TestClient tests.

Acceptance:

make test tests/api passes

## Registry Agent Prompt

Build model, policy, and tenant/public alias registries.

Support YAML mode first.

Support Supabase-backed loading interface.

Add:

model\_registry.yaml

policy\_registry.yaml

tenant\_config.yaml

Acceptance:

make test tests/registries passes

## Executor Agent Prompt

Build mock executor and Portkey/OpenRouter executor.

Normal tests must mock HTTP.

Real provider smoke test must be gated behind RUN\_REAL\_PROVIDER\_TESTS=true.

Acceptance:

make test tests/executor passes

RUN\_REAL\_PROVIDER\_TESTS=true make smoke-real works when keys are present

## Storage Agent Prompt

Build Supabase storage layer and SQL schema.

Normal tests must use mocked Supabase client.

Real Supabase tests must be gated behind RUN\_SUPABASE\_TESTS=true.

Acceptance:

make test tests/storage passes

RUN\_SUPABASE\_TESTS=true make test tests/storage works when env vars are present

## Baseline Policy Agent Prompt

Build baseline RouterPolicy plug-ins.

Implement:

```
AlwaysStrongestPolicy
AlwaysCheapestPolicy
AlwaysFastestPolicy
ManualRulesPolicy
RandomPolicy
```

No external APIs.

Acceptance:  
make test tests/policies passes

## Runtime Agent Prompt

Build RuntimeKernel and RoutePlanValidator.

Connect:  
API request  
request normalizer  
public model resolver  
candidate resolver  
policy loader  
RoutePlan validator  
executor  
response normalizer  
trace writer

Use mock executor for local tests.

Acceptance:  
make test tests/runtime passes  
make smoke-mock passes

## Benchmark Agent Prompt

Build benchmark and shadow testing system.

Implement:  
mock-benchmark  
dry-run-benchmark  
canary-real-benchmark  
full-benchmark with cost confirmation guard

Do not run full benchmarks automatically.  
Canary real benchmark must have:

```
--limit-per-dataset
--max-real-calls
--max-cost-usd
--timeout-seconds
```

Acceptance:

```
make test tests/evals passes
make benchmark-mock passes
make benchmark-dry-run passes
RUN_REAL_PROVIDER_TESTS=true make benchmark-canary-real works when keys are
present
```

## Deployment/Docs Agent Prompt

Build deployment and docs.

Add:

```
Dockerfile
render.yaml
.env.example
README.md
docs/customer-usage.md
docs/testing.md
docs/benchmarking.md
docs/deployment.md
docs/add-new-model.md
docs/add-new-policy.md
```

Acceptance:

```
Docker build passes
make check passes
docs explain all required env vars and test modes
```

---

## 20. Final Acceptance Checklist

The product is not done until all are true:

```
[ ] make install passes
[ ] make lint passes
[ ] make typecheck passes
[ ] make test passes
[ ] make smoke-mock passes
```



```
[ ] make benchmark-mock passes
[ ] make benchmark-dry-run passes

[ ] /health works
[ ] /v1/models works
[ ] /v1/chat/completions works in mock mode
[ ] mock response preserves public model name
[ ] RoutePlan is generated and validated
[ ] invalid RoutePlan does not execute
[ ] trace object is created
[ ] Supabase schema exists
[ ] Supabase integration test works when enabled
[ ] Portkey/OpenRouter smoke test works when enabled
[ ] canary real benchmark works when enabled
[ ] full benchmark cannot run accidentally
[ ] Render config exists
[ ] docs explain how to add a new model
[ ] docs explain how to add a new policy
```

---

## 21. Final End-to-End Test

Run in this order:

```
make check
```

Then, with Supabase env vars:

```
RUN_SUPABASE_TESTS=true make test
```

Then, with Portkey/OpenRouter keys:

```
RUN_REAL_PROVIDER_TESTS=true make smoke-real
```

Then, cheap real benchmark canary:

```
RUN_REAL_PROVIDER_TESTS=true make benchmark-canary-real
```

Do not run full benchmark during MVP development.

---

## 22. Final Instruction

Build the runtime shell first.

The final architecture must make this true:

```
New model:  
  add model string/config only  
  
New router:  
  implement RouterPolicy.plan() only  
  
New customer-facing model:  
  add public alias mapping only  
  
New benchmark:  
  add dataset/config only
```

Normal development must be cheap:

```
unit tests: $0  
mock smoke tests: $0  
dry-run benchmarks: $0  
real smoke tests: tiny  
canary real benchmarks: capped to cents  
full benchmarks: manual only
```

Final customer behavior:

```
Customer calls:  
  model="brainbase-chat"  
  
Runtime:  
  resolves alias  
  loads policy  
  produces RoutePlan  
  validates RoutePlan  
  executes selected model through Portkey/OpenRouter  
  returns OpenAI-compatible response  
  stores trace  
  supports safe benchmarking and shadow testing
```